

# 014: StepPPO-v3 Method

2026-06-09

本文定义 StepPPO-v3 的方法版本。v3 的目标不是继续在 StepPPO-v2 上调一个权重, 而是把 multi-turn agent 训练的三个关键粒度对齐:

1. 环境粒度: 一个 WebShop step 的完整 response 是一个 action。
2. 信用分配粒度: advantage / return 在 environment step 轴上计算。
3. 优化约束粒度: PPO ratio / clipping 在 response/action 级别计算。

同时, v3 默认把 value loss 变成 critic-safe auxiliary path: value head 可以学习 step value, 但 value loss 不回传 actor backbone。

## 1. Motivation

已有结果显示, StepPPO-v1/v2 和 GiGPO value-aux 都还没有超过 GiGPO baseline。最关键的诊断来自 GiGPO-v1 value-aux detach=false: policy advantage 保持 GiGPO 不变, 只额外加入 shared value regression, 仍然出现明显退化和 step 178–185 validation collapse。这说明失败不只来自 mixed advantage, 也可能由 value target 的高方差和 shared-backbone gradient 干扰引入。

StepPPO-v3 因此采用以下设计原则:

- 不再混入 raw episode advantage, 避免 episode reward 口径和 step reward 口径不一致。
- 所有 step advantage、value return、invalid action penalty 使用同一套 shaped step reward。
- value loss 默认不更新 actor backbone, 先验证 step credit assignment 本身是否有用。
- policy update 使用 action-level sequence ratio, 避免同一个 environment action 内 token 被不一致 clipping。
- 保留 GiGPO 的 group-relative 稳定性, 但把归一化作用于 step advantage, 而不是再加一个 episode term。

## 2. Agentic Step MDP

一个任务 rollout 由多个 environment steps 构成:

$$\text{tau} = (\text{s}_0, \text{a}_0, \text{r}_0, \text{s}_1, \text{a}_1, \text{r}_1, \dots, \text{s}_{\{T-1\}}, \text{a}_{\{T-1\}}, \text{r}_{\{T-1\}})$$

其中  $\text{s}_t$  是执行第  $t$  个 agent action 前的上下文,  $\text{a}_t$  是语言模型生成的完整 response/action string,  $\text{r}_t$  是该 step 的 shaped reward。对 WebShop 来说, 环境执行的是完整 action, 例如:

```
search[wireless mouse]
click[Buy Now]
```

环境不会分别执行 response 内部的 token。因此 v3 把 `a_t` 作为一个 environment-level action，而不是把 response token 当成 MDP step。

每个 step sample `i` 存储三个关键 id:

```
uid:                task / prompt    rollout group
traj_uid:           sampled trajectory
step_idx: trajectory    environment step index
```

GAE 在 `traj_uid` 内按 `step_idx` 排序后计算；group normalization 在同一 `uid` 内计算。

### 3. Shaped Step Reward

v3 只使用一套训练 reward 口径。对每个 step sample `i`:

```
r_i^v3 = raw_env_reward_i
        - invalid_action_penalty_coef * 1[is_action_valid_i = false]
        + kl_reward_delta_i
```

其中 `kl_reward_delta_i` 只在 `algorithm.use_kl_in_reward=True` 时加入:

```
kl_reward_delta_i = sum(token_level_rewards_i - token_level_scores_i)
```

v3 不再优先读取 raw episode\_rewards 来构造 policy advantage。这样可以避免 v2 中的口径分裂:

```
A_epi    raw episode reward
A_step    invalid-action-penalized step reward
```

v3 的 policy advantage 和 value target 都来自 `r_i^v3`。

### 4. State Value Head

v3 沿用 shared actor 上的 step value head，但默认设置:

```
actor_rollout_ref.actor.value_head.detach_value_backbone: true
```

给定 step sample `i` 的 tokenized context-response sequence，actor backbone 产生 hidden states:

```
H_i = f_theta(s_i, a_i)
```

value position 使用 action 前最后一个 context token:

```
b_i = full_sequence_length_i - response_length_i - 1
V_i = v_psi(stop_grad(H_i[b_i]))
```

这里 `stop_grad` 只用于 value loss 路径。policy gradient 仍然正常更新 actor backbone。这样做的含义是:

- value head 学习当前 actor hidden state 上的 step value probe。
- value loss 只更新 value head 参数 `psi`。

- actor backbone 不会被高方差 value regression 直接牵引。

如果后续 `detach=true` 稳定但收益不足，再单独测试 `detach=false` 或 `separate critic`，而不是把它作为 v3 默认。

## 5. Step GAE

在每次 PPO update 前，先用 rollout policy 对 batch 计算 old state values:

```
V_i^old = V_{theta_old, psi_old}(s_i)
```

对 trajectory 内 step sample  $i=(\tau, t)$ ，若下一步存在，记为  $i+ = (\tau, t+1)$ ；若当前 step 是终止 step，则 `done_i = 1`。TD residual:

```
delta_i = r_i^v3 + gamma * (1 - done_i) * V_{i+}^old - V_i^old
```

GAE 在 environment-step 轴上反向递推:

```
A_i^raw = delta_i + gamma * lambda * (1 - done_i) * A_{i+}^raw
```

value regression target:

```
R_i^step = A_i^raw + V_i^old
```

v3 默认超参:

```
algorithm.step_ppo_v3.step_gamma: 0.95
algorithm.step_ppo_v3.step_lam: 0.90
```

`gamma=0.95` 沿用当前 WebShop/GiGPO 口径；`lambda=0.90` 是为了降低 pure Monte Carlo return 的方差。正式实验需要 ablate `lambda` in `{0.80, 0.90, 0.95, 1.00}`。

## 6. Group-Normalized Step Advantage

v3 不再构造 `A_epi`。最终 policy advantage 直接来自 step GAE，并在同一 rollout group 内标准化。

对同一个 `uid=u` 的 step sample 集合:

```
I_u = { i | uid_i = u }
```

计算:

```
mu_u = mean({ A_i^raw | i in I_u })
std_u = std({ A_i^raw | i in I_u })
```

最终 step advantage:

```
A_i^v3 = (A_i^raw - mu_uid(i)) / (std_uid(i) + eps)
```

如果某个 group 的 valid sample 数不足或 `std_u` 接近 0，该 group 的 policy advantage 置零或跳过 policy loss，只保留 value diagnostics。这和 GRPO/GiGPO 的基本稳定性来源一致：只优化同一 task group 内有相对差异的 rollout。

## 7. Action-Level PPO Ratio

v3 的核心变化是：一个 environment step 的完整 response 是 action，因此 PPO clipping 也在 response/action 级别做。

对 step sample  $i$  的 response tokens，定义 token log-ratio：

```
d_{i,l} = log pi_theta(y_{i,l} | s_i, y_{i,<l})
          - log pi_old(y_{i,l} | s_i, y_{i,<l})
```

真实 action probability ratio 是：

```
rho_i^sum = exp(sum_l m_{i,l} * d_{i,l})
```

但长 response 上  $\rho_i^{\text{sum}}$  很容易指数爆炸。v3-core 采用工程上更稳的 sequence geometric ratio：

```
L_i = sum_l m_{i,l}
rho_i = exp((1 / L_i) * sum_l m_{i,l} * d_{i,l})
```

这和当前代码里已有的 `compute_policy_loss_gspo` 思路一致：一个 response 内所有 token 共享同一个 sequence-level ratio。它不是把 token 各自独立 clip，而是用整段 response 的平均 log-ratio 决定这个 action 是否超过 PPO trust region。

PPO clipped objective：

```
L_pg^v3(theta)
= - mean_i [
    min(
        rho_i * A_i^v3,
        clip(rho_i, 1 - eps_low, 1 + eps_high) * A_i^v3
    )
]
```

实现上可以把  $\rho_i$  broadcast 到 response tokens，再用 `seq-mean-token-mean` 聚合，保证每条 response/action 贡献相同权重。

## 8. Value Loss

v3 的 value loss 是 step-level scalar loss，而不是 token-level broadcast loss：

```
V_i(theta, psi) = v_psi(stop_grad(H_i[b_i]))
```

clipped value prediction：

```
V_i^clip = V_i^old + clip(V_i - V_i^old, -eps_v, eps_v)
```

value loss：

```
L_v(psi)
= 0.5 * mean_i max(
    (V_i - R_i^step)^2,
    (V_i^clip - R_i^step)^2
)
```

v3 默认：

```
actor_rollout_ref.actor.value_head.value_loss_coef: 0.03
actor_rollout_ref.actor.value_head.cliprange_value: 0.5
```

因为 `detach_value_backbone=true`, 该 loss 不直接更新 actor backbone。若 value head 仍不稳定, 再测试:

```
value_loss_coef: 0.01
value_loss_type: huber
normalize_value_target: true
```

## 9. Total Loss

v3 的总 loss:

```
L_v3(theta, psi)
= L_pg^v3(theta)
+ c_v * L_v(psi)
+ c_kl * L_kl(theta)
- c_H * H(theta)
```

其中默认仍可保持当前 WebShop 设定:

```
algorithm.use_kl_in_reward: false
entropy_coeff: 0
```

## 10. Training Iteration

Input:

```
actor pi_theta with value head v_psi
grouped rollout tasks
step_gamma gamma
step_lam lambda
```

1. Rollout:

```
collect K trajectories per uid.
store uid, traj_uid, step_idx, raw rewards, is_action_valid.
```

2. Build shaped step reward:

```
r_i^v3 = raw reward - invalid penalty + optional KL reward delta.
```

3. Old-policy evaluation:

```
compute old_log_prob for response tokens.
compute old state value V_i^old at pre-action boundary.
```

4. Step GAE:

```
group by traj_uid.
sort by step_idx.
compute delta_i, A_i^raw, R_i^step.
```

```

5. Group normalization:
    group A_i^raw by uid.
    compute A_i^v3.
    drop or zero no-variance groups.

6. PPO update:
    recompute current log_prob and current state values.
    compute sequence geometric ratio rho_i.
    apply response/action-level PPO clipping with scalar A_i^v3.
    compute scalar clipped value loss against R_i^step.
    update actor by policy loss; update value head by value loss.

```

## 11. Primary Configuration

建议 v3-core 先使用以下配置:

```

algorithm:
  adv_estimator: step_ppo_v3
  gamma: 0.95
  step_ppo_v3:
    reward_source: shaped_step
    use_episode_advantage: false
    step_gamma: 0.95
    step_lam: 0.90
    advantage_norm: uid
    zero_no_variance_group: true
    policy_ratio: sequence_geometric
    final_advantage_mode: pure_step

actor_rollout_ref:
  actor:
    policy_loss: action_level
    loss_agg_mode: seq-mean-token-mean
    value_head:
      enable: true
      value_position: pre_action_last_context_token
      detach_value_backbone: true
      value_loss_coef: 0.03
      cliprange_value: 0.5

```

## 12. Required Diagnostics

v3 必须新增以下指标, 否则无法判断失败来自 value、advantage 还是 policy ratio:

```

actor/action_ratio_mean
actor/action_ratio_std
actor/action_clipfrac

```

```

actor/action_approx_kl

actor/value_rmse
actor/value_mae
actor/value_explained_variance
actor/value_pred_std
actor/value_return_std
actor/value_return_p05
actor/value_return_p50
actor/value_return_p95

critic/step_adv_raw_mean
critic/step_adv_raw_std
critic/step_adv_norm_mean
critic/step_adv_norm_std
critic/no_variance_group_ratio

episode/valid_action_ratio
response_length/clip_ratio

```

还应记录 valid vs invalid action 分层：

```

critic/step_adv_valid_mean
critic/step_adv_invalid_mean
actor/value_loss_valid
actor/value_loss_invalid

```

这些指标用于回答三个问题：

1. value head 是否真的有 predictive signal。
2. invalid action penalty 是否造成 return outlier。
3. action-level ratio 是否比 token-level ratio 更好地约束 response format。

## 13. Ablation Order

v3 的最小可验证路线：

1. v3\_core: pure step GAE + uid advantage norm + action-level ratio + detach value backbone.
2. v3\_no\_action\_ratio: 只把 policy loss 换回 token-level PPO, 确认 action-level ratio 的贡献。
3. v3\_detach\_false: 只把 value loss 回传 backbone, 确认 shared-backbone interference 是否仍存在。
4. v3\_lam\_1: 将 step\_lam=1.0, 确认 v2/v1 的高方差是否来自 Monte Carlo-style target。
5. v3\_batch\_norm\_adv: 将 advantage\_norm=uid 改为 batch, 确认 group normalization 是否必要。

首要成功标准不是马上超过 GiGPO, 而是：

```
valid_action_ratio      GiGPO
response_clip_ratio      GiGPO
value loss              validation collapse
step 200 task/success/text      StepPP0-v2
```

在满足这些稳定性条件后, 才值得扩展到 separate critic LR、Huber value loss、in-distribution critic pretraining 或 GAGPO-style grouped value proxy。